

软件23 《移动应用开发》课程考查大作业方案

📄 方案概览

🎯 考核目标

本考核方案旨在全面评估学生在移动应用开发领域的**技术深度、跨平台整合能力、团队协作水平和工程实践能力**。通过真实的项目场景，采用 AI 新范式模板化开发流程（需求分析→AI 辅助编码→测试验证→部署运维），培养学生解决复杂技术问题的能力，为进入移动开发行业做好充分准备。

📊 考核特色

- 🌐 **全技术栈覆盖**：涵盖 Android/iOS/Flutter/React Native/Uniapp/MAUI/鸿蒙/小程序七大技术栈
- 👥 **团队协作模式**：每组 6 人，按项目业务组+开发技术组模式，模拟企业级开发团队分工
- 🤖 **AI 新范式驱动**：全流程采用"需求→编码→测试→运维"四阶段模板化开发，使用 TRAE AI 辅助管理
- 🔗 **后端交互整合**：统一 RESTful API 接口，考察前后端数据交互能力
- 📶 **硬件与物联网**：融入传感器调用与物联网数据采集场景
- 🔐 **Git 协作规范**：使用 Gitee 进行版本控制与团队协作
- ⚠️ **鸿蒙必须技术栈**：所有同学必须掌握鸿蒙开发技术
- 📝 **AI 批阅环节**：学生提交项目代码后，AI 会自动批阅，提供反馈和建议
- 🔄 **项目管理环节**：学生需要在项目中进行有效的沟通和合作，确保项目按计划进行
- 🏆 **项目展示环节**：学生需要在项目完成后，展示项目功能和性能，确保项目符合要求

📊 考核成绩构成

考核项	比例	说明
平时成绩	20%	作业（50%）+ 课堂表现（20%）+ 阶段测验（30%）
实验成绩	30%	实验完成情况 + 个人技术栈深度 + 组内分享质量 + 实验报告
期末考查	50%	大作业：多端应用开发 + 技术选型报告 + 项目文档

🏆 考核成果

- 🚀 **可运行的多端应用系统**：包含 6-8 个不同技术栈的客户端
- 🔗 **统一的后端 API 服务**：支持多端数据同步和业务逻辑
- 📄 **完整的技术文档**：需求文档、架构设计、技术选型对比报告、测试报告（采用 Markdown 格式，PDF 提交）
- 📁 **Gitee 项目仓库**：规范的分支管理与提交记录，方便学生查看和管理项目代码
- 📊 **PlantUML 项目模型图**：使用 PlantUML 绘制的项目架构图、类图、顺序图等
- 📝 **AI 开发过程记录**：体现 TRAE 在各阶段的辅助作用
- 🗄️ **SQLite 数据库**：存储用户数据、需求数据、评价数据等
- ☁️ **Firebase 云服务**：提供后端支持，包括用户认证、数据库存储、消息通知等
- 🔄 **数据同步机制**：跨平台数据同步，确保用户在不同设备上的数据一致性；离线在线数据同步，确保用户在不同网络环境下的数据一致性

项目分组模式

项目业务组（选题方向）

每届可选 3-4 个项目方向，各组选择一个：

组别	项目方向	主要功能
智慧校园	校园服务	通知公告、课表查询、图书借阅
智慧健康	运动健康	步数统计、心率监测、运动记录
智慧生活	生活助手	天气查询、闹钟提醒、备忘录
智慧购物	电商应用	商品列表、购物车、订单管理
需求匹配平台	服务对接	用户注册、登录、个人中心、发布需求、需求匹配、需求完成、评价反馈

 **注意：**所有项目方向都必须包含以下基本业务逻辑：

- **用户注册：**支持多种注册方式（手机号、邮箱、第三方登录）
- **用户登录：**支持多种登录方式，记住密码，自动登录
- **个人中心：**个人信息管理、设置修改、数据统计
- **发布需求：**用户可以发布自己的需求，填写需求详情
- **需求匹配：**系统自动匹配相关需求，推荐给合适的用户
- **需求完成：**需求完成后进行确认和结算
- **评价反馈：**用户可以对需求完成情况进行评价和反馈

开发技术组（技术分工）

每组 6 人，每人负责一个技术栈：

序号	技术栈	开发语言	开发工具	必选
1	Android	Kotlin	Android Studio	✓
2	iOS	Swift/SwiftUI	Xcode	
3	Flutter	Dart	VS Code	✓
4	React Native	JavaScript/JSX	VS Code	✓
5	Uniapp	Vue.js	HBuilderX	
6	MAUI	C#	Visual Studio	
7	鸿蒙	ArkTS/ArkUI	DevEco Studio	✓
8	微信小程序	JavaScript	微信开发者工具	✓

 **注意：**Android、Flutter、React Native、鸿蒙、微信小程序为必选技术栈，每位同学至少掌握其中一个。

题目一：智慧校园生活服务平台

📖 项目概述

项目名称：SmartCampus - 智慧校园生活服务平台 **难度等级：**★★★★（中高难度） **开发周期：**15 天 **团队规模：**6 人小组

项目背景：随着数字化校园建设的深入，学生对一站式校园服务的需求日益增长。本项目旨在开发一个集信息获取、服务办理、生活便民于一体的多端校园服务平台。

🎯 核心业务模块

模块一：校园资讯中心 📰

- 通知公告：学校官方通知、紧急公告推送
- 校园动态：校园新闻、活动资讯、学术讲座
- 院系新闻：各院系最新动态
- 个性化推荐：基于用户兴趣的内容推荐

模块二：校园服务中心 🏢

- 失物招领：物品发布、搜索匹配、联系对接
- 校园报修：设施报修、进度跟踪、满意度评价
- 场馆预约：图书馆、体育馆、会议室预约管理
- 请假申请：学生请假流程管理

模块三：生活便民工具 🗺️

- 校园地图导航：GPS 定位、路径规划（传感器集成）
- 校车时刻表：实时班次、到站提醒
- 作息日历：学校作息时间表查询
- 教室查询：空闲教室、占用状态

模块四：个人中心 👤

- 我的预约、消息通知、个人资料、系统设置

👥 技术团队分工

角色	技术栈	核心职责	特色功能
Android 开发	Kotlin + Jetpack	原生性能优化	推送服务、GPS 定位、传感器集成
Flutter 开发	Dart + Flutter	跨平台统一	状态管理、动画效果、传感器插件
React Native 开发	JavaScript + React Native	JS 生态	原生模块桥接、热更新机制
鸿蒙开发	ArkTS + ArkUI	多端部署	分布式能力、传感器调用
Uniapp 开发	Vue + Uniapp	多端编译	条件编译、H5+小程序双端
微信小程序	JavaScript	轻量应用	免安装、即用即走

🔗 技术参考方案

基础要求

各平台实现核心功能，遵循统一设计规范（含颜色体系、组件样式、交互逻辑），保证基础 UI 一致性。

平台特色技术

- **Android 端**：集成华为 / 小米推送服务，实现后台消息唤醒；使用 Jetpack 组件（ViewModel、Room）优化数据管理；冷启动时间 < 3 秒。
- **iOS 端**：适配暗黑模式与动态岛（iPhone 14+）快捷操作；使用 SwiftUI 构建响应式界面；集成 iOS 共享菜单实现资讯快速分享。
- **Flutter 端**：实现与原生模块通信（如调用 Android/iOS 地图 SDK）；使用 Bloc 模式管理全局状态；优化列表滑动性能（≥60fps）。
- **Cordova 端**：开发 2 个自定义插件（校园地图定位插件、设备信息获取插件）；使用 Cordova CLI 实现自动化构建；通过 `cordova-plugin-whitelist` 解决跨域问题。
- **Xamarin 端**：采用 MVVM 模式架构；集成 Xamarin.Essentials 访问设备传感器（如定位）；实现与 .NET 后端（ASP.NET Core）的数据交互。
- **微信小程序**：接入微信授权登录；实现场馆预约的微信支付模拟；使用小程序云开发存储用户发布的失物招领信息。
- **鸿蒙端**：利用 ArkUI 声明式 UI 构建响应式界面；实现分布式能力（如多设备协同）；集成鸿蒙传感器 API 实现位置和环境感知。

整合要求

1. 搭建统一后端 API（推荐 ASP.NET Core 或 Node.js），实现用户数据（登录状态、预约记录等）多端同步。
2. 开发 "平台入口" 页面，提供各端应用的下载 / 访问链接及功能对比说明。
3. 实现跨平台数据互通：用户在任意端发布的信息（如失物招领），其他端实时可见。
4. 跨框架交互：Cordova 应用通过 JavaScript 桥接调用 Xamarin 的文件处理模块（如导出报修记录为 PDF）。
5. 鸿蒙应用与其他平台的协同：利用鸿蒙分布式能力，实现与 Android/Flutter 应用的数据共享和功能协同。

AI 新范式开发流程

阶段一：需求分析（第 1-2 天）

- 团队协作完成项目需求文档（功能列表、用户故事、数据模型）
- 使用 TRAE 辅助生成 API 接口文档与数据库设计
- 确定统一的 RESTful API 规范与 JSON 数据格式
- 制定 Git 分支策略与协作规范

阶段二：AI 辅助编码（第 3-10 天）

- 各成员使用 TRAE 辅助各技术栈端的代码开发
- 统一对接后端 RESTful API（Token 认证、JSON 解析）
- 集成设备传感器（GPS 定位用于校园地图、加速度计用于运动检测）
- 每日 Git 提交，保持代码同步

阶段三：测试验证（第 11-13 天）

- 制定测试计划，编写测试用例
- 功能测试：核心业务流程验证
- 兼容性测试：跨端数据同步与 UI 一致性
- 性能测试：启动速度、网络请求响应、传感器数据采集

阶段四：部署运维（第 14-15 天）

- 多端应用打包部署
- 撰写技术选型对比报告（各框架优劣分析）
- 编写项目总结文档
- 团队答辩演示

 技术实现要求

统一设计规范：

- UI 设计系统：统一的色彩规范、字体规范、组件库
- 交互规范：一致的导航模式、手势操作、反馈机制
- 响应式设计：适配不同屏幕尺寸

后端 API 要求：

- RESTful 风格接口设计
- JSON 数据格式统一
- Token 认证机制
- 错误码规范

Git 协作要求：

- main + dev + feature/xxx 分支策略
- 规范的 commit message
- 组内代码审查（Code Review）

 题目二：健康运动管理平台

 项目概述

项目名称：FitLife - 健康运动管理平台 **难度等级：**★★★★（中高难度） **开发周期：**15 天 **团队规模：**6 人小组

项目背景：随着健康意识的提升，人们对运动数据记录和健康管理的需求日益增长。本项目充分利用移动设备传感器能力，开发一个集运动记录、健康数据分析、社交互动于一体的多端健康管理平台。

 核心业务模块

模块一：运动记录

- 步数统计：加速度传感器实时计步

- 运动轨迹：GPS 定位记录运动路线
- 运动计时：多种运动模式（跑步/骑行/游泳）计时
- 卡路里估算：基于运动数据的热量消耗计算

模块二：健康数据 🏠

- 数据仪表盘：运动数据可视化展示
- 历史记录：按日/周/月查看运动趋势
- 健康报告：定期生成健康分析报告
- 目标设定：每日运动目标与达成提醒

模块三：社交互动 🧑🏻

- 运动排行榜：好友间运动数据排名
- 运动打卡：每日打卡与分享
- 运动圈：动态发布与互动评论

模块四：个人中心 🧑

- 个人资料、运动偏好设置、设备绑定、数据同步

🧑🏻 技术团队分工

角色	技术栈	核心职责	传感器/硬件特色
Android 开发	Kotlin + Jetpack	原生传感器深度集成	加速度计计步、GPS 轨迹、陀螺仪姿态检测
iOS 开发	Swift/SwiftUI	健康数据集成	HealthKit 数据同步、Siri 快捷指令
Flutter 开发	Dart + Flutter	跨平台数据可视化	sensors_plus 插件、geolocator 插件
React Native 开发	JavaScript + React Native	社交互动模块	react-native-sensors、地理位置 API
鸿蒙开发	ArkTS + ArkUI	多端与分布式	传感器 API、分布式数据同步
Uniapp 开发	Vue + Uniapp	多端运动打卡	uni.getLocation、uni.onAccelerometerChange
微信小程序	JavaScript	轻量社交	微信运动、分享功能

🔗 技术参考方案

基础要求

各平台支持运动数据采集（自动 + 手动录入）、数据展示与基础社交功能，支持离线记录与联网同步。

平台特色技术

- **Android 端**：调用计步传感器与 GPS 实现自动轨迹记录；集成 Google Fit 同步数据；使用 WorkManager 实现后台持续计步。
- **iOS 端**：接入 HealthKit 获取系统运动数据；实现 Siri 快捷指令（如 "开始跑步"）；使用 Core Location 优化轨迹精度。
- **Flutter 端**：使用charts_flutter绘制运动趋势图表；通过geolocator插件获取定位；使用 Hive 数据库实现离线数据缓存。
- **Cordova 端**：使用cordova-plugin-device-motion获取加速度数据；开发后台持续计步插件（解决 WebView 休眠问题）；集成cordova-plugin-health同步系统健康数据。
- **MAUI 端**：实现 Windows + 移动设备跨平台部署；使用.NET MAUI Charts 绘制数据图表；集成系统健康服务（Android Health Connect、iOS HealthKit）；通过 Blazor Hybrid 混合 Web 与原生控件。
- **Uniapp 端**：通过条件编译适配 H5 与支付宝小程序；使用支付宝运动 API 同步步数；通过云函数实现排行榜实时计算。
- **鸿蒙端**：利用鸿蒙传感器框架实现高精度计步；通过分布式能力实现多设备运动数据同步；使用 ArkUI 绘制运动数据可视化图表。

整合要求

1. 实现运动数据多端同步（支持离线记录，联网后自动增量同步），设计数据冲突解决机制（如同一时段多端数据合并规则）。
2. Cordova 应用与 MAUI 应用共享.NET 后端的运动数据分析服务（如卡路里计算、运动强度评估）。
3. 开发统一数据看板，对比各平台数据采集精度（如 GPS 轨迹偏差、计步误差）。
4. 实现跨平台社交互动：Uniapp 用户可查看 Flutter 端用户的运动动态并点赞。
5. 鸿蒙应用与其他平台的协同：利用鸿蒙分布式能力，实现运动数据在多设备间的实时同步。

AI 新范式开发流程

(同题目一四阶段流程，重点突出传感器数据采集与健康数据分析)

题目三：智能家居控制平台

项目概述

项目名称：SmartHome - 智能家居控制平台 **难度等级：**★★★★★（高难度） **开发周期：**15 天 **团队规模：**6 人小组

项目背景：物联网技术快速发展，智能家居成为重要应用场景。本项目模拟智能家居控制场景，开发多端控制平台，重点考察移动设备与物联网设备的数据交互能力。

核心业务模块

模块一：设备管理

- 设备列表：智能灯光、空调、窗帘、安防摄像头等
- 设备状态：实时状态监控与展示
- 设备控制：远程开关、参数调节
- 设备分组：按房间/场景分组管理

模块二：环境监测

- 温湿度监测：模拟温湿度传感器数据展示
- 光照强度：利用手机光线传感器模拟环境光检测
- 空气质量：模拟 PM2.5 等数据展示
- 数据趋势：历史环境数据图表分析

模块三：智能场景 🏠

- 场景模式：回家模式、离家模式、睡眠模式
- 定时任务：设备定时开关控制
- 联动规则：基于传感器数据触发设备联动（如光线暗→开灯）

模块四：安全中心 🛡️

- 异常告警：设备异常、环境异常推送
- 操作日志：设备操作历史记录
- 权限管理：家庭成员权限分配

👥 技术团队分工

角色	技术栈	核心职责	物联网特色
Android 开发	Kotlin + Jetpack	设备控制核心	光线传感器联动、蓝牙模拟通信
Flutter 开发	Dart + Flutter	环境数据可视化	传感器数据实时图表展示
React Native 开发	JavaScript + React Native	设备列表与控制	WebSocket 实时通信
鸿蒙开发	ArkTS + ArkUI	分布式控制	鸿蒙分布式软总线、多设备协同
Uniapp 开发	Vue + Uniapp	多端场景配置	条件编译适配 H5/小程序
微信小程序	JavaScript	轻量控制	模板消息、扫码控制

🔗 技术参考方案

基础要求

各平台实现设备管理、环境监测、智能场景控制等核心功能，支持与模拟物联网设备的交互。

平台特色技术

- **Android 端**：利用光线传感器实现智能灯光控制；通过蓝牙模拟与智能设备通信；使用 Jetpack Compose 构建现代化界面。
- **iOS 端**：利用 Core Bluetooth 模拟智能设备连接；实现 Siri 语音控制家居设备；使用 SwiftUI 构建响应式控制界面。
- **Flutter 端**：使用flutter_blue插件模拟蓝牙设备通信；通过charts_flutter实现环境数据可视化；使用 Provider 进行状态管理。
- **React Native 端**：使用react-native-ble-plx模拟蓝牙通信；通过 WebSocket 实现实时设备状态更新；使用 Redux 管理全局状态。
- **鸿蒙端**：利用鸿蒙分布式软总线实现多设备协同控制；通过 ArkUI 构建卡片式控制界面；集成鸿蒙传感器 API 实现环境数据采集。

- **Uniapp 端**：通过条件编译适配 H5 与小程序；使用 uni.getBLEDeviceServices 模拟蓝牙设备交互；通过云函数实现远程控制。
- **微信小程序**：实现扫码控制设备功能；使用模板消息推送设备状态通知；通过小程序云开发存储设备配置。

 整合要求

1. 实现设备状态多端实时同步（基于 WebSocket），确保各端设备状态一致。
2. 开发统一的设备管理后台，支持添加、删除、配置智能设备。
3. 实现跨平台场景联动：如 Android 端设置的 "回家模式"，其他端自动同步执行。
4. 鸿蒙应用与其他平台的协同：利用鸿蒙分布式能力，实现多设备协同控制场景。
5. 设计统一的设备通信协议，确保各平台与模拟物联网设备的交互一致性。

 AI 新范式开发流程

(同题目一四阶段流程，重点突出物联网数据交互与传感器联动逻辑)

 题目四：多端新闻资讯聚合平台

 项目概述

项目名称：NewsHub - 多端新闻资讯聚合平台 **难度等级**：★★★（中等难度） **开发周期**：15 天 **团队规模**：6 人小组

项目背景：信息时代用户需要高效获取和管理新闻资讯。本项目开发多端新闻聚合平台，重点考察 RESTful API 数据交互、多端 UI 适配与内容展示能力。

 核心业务模块

模块一：新闻浏览

- 分类浏览：科技/财经/体育/娱乐等频道
- 新闻详情：富文本展示、图片/视频加载
- 搜索功能：关键词搜索、历史记录
- 下拉刷新与分页加载

模块二：个性化推荐

- 兴趣标签：用户兴趣偏好设置
- 阅读历史：浏览记录与阅读时长统计
- 智能推荐：基于标签的内容推荐

模块三：互动功能

- 收藏夹：新闻收藏与分类管理
- 评论系统：新闻评论与回复
- 分享功能：多平台分享

模块四：个人中心

- 阅读统计、偏好设置、夜间模式、字体大小调节

👥 技术团队分工

角色	技术栈	核心职责	特色功能
Android 开发	Kotlin + Jetpack	新闻列表与详情	RecyclerView 优化、离线缓存
Flutter 开发	Dart + Flutter	跨平台统一体验	自定义主题、动画过渡
React Native 开发	JavaScript + React Native	互动功能模块	FlatList 优化、实时评论
鸿蒙开发	ArkTS + ArkUI	多端适配	分布式数据、跨设备阅读
Uniapp 开发	Vue + Uniapp	H5+小程序双端	条件编译、分享接口适配
微信小程序	JavaScript	轻量阅读	订阅消息、收藏功能

🔗 技术参考方案

基础要求

各平台实现新闻浏览、个性化推荐、互动功能等核心功能，支持 RESTful API 数据交互。

平台特色技术

- **Android 端**：使用 RecyclerView 优化新闻列表加载性能；实现视频离线下载与加密播放；支持悬浮窗学习（Picture-in-Picture）。
- **iOS 端**：支持画中画播放与 Split View 分屏阅读；接入 ARKit 实现 3D 新闻展示；使用 Core Data 管理离线资源。
- **Flutter 端**：使用webview_flutter集成在线内容；通过sqflite实现新闻离线缓存；使用video_player优化视频播放体验。
- **React Native 端**：使用 FlatList 优化新闻列表性能；通过react-native-video实现视频播放；使用 Redux 管理全局状态。
- **鸿蒙端**：利用 ArkUI 声明式 UI 构建响应式新闻界面；通过分布式能力实现跨设备阅读同步；使用鸿蒙网络能力优化 API 请求。
- **Uniapp 端**：通过条件编译适配 H5 与小程序；使用 uni.request 实现 API 数据交互；通过云函数实现个性化推荐。
- **微信小程序**：实现新闻分享与订阅功能；使用订阅消息推送新闻更新；通过小程序云开发存储用户阅读记录。

🔗 整合要求

1. 实现用户阅读进度多端同步（如 Android 端阅读的新闻，其他端自动标记为 "已读"）。
2. 开发统一的新闻管理后台，支持各平台新闻数据更新。
3. 实现跨平台评论系统：用户在任意端发表的评论，其他端实时可见。
4. 鸿蒙应用与其他平台的协同：利用鸿蒙分布式能力，实现新闻阅读状态在多设备间的同步。
5. 设计统一的 RESTful API 接口规范，确保各平台数据交互的一致性。

🔄 AI 新范式开发流程

(同题目一四阶段流程，重点突出 RESTful API 交互与多端 UI 适配)

评分标准

总体评分维度

评分维度	权重	评价要点
功能完整性	25%	核心模块实现度，无关键功能缺失；边缘场景（如网络异常）处理合理性
技术深度	25%	各平台特色功能实现质量（如 Cordova 插件完整性、MAUI 跨设备适配效果、鸿蒙分布式能力）
跨框架整合效果	20%	数据同步一致性（≤500ms 延迟）；跨框架交互流畅度；多端协同体验
AI 新范式执行	10%	四阶段流程完整性、TRAE 使用记录、文档规范
团队协作	10%	Git 提交规范、代码审查记录、分工合理性
技术选型报告	5%	框架对比分析深度、选型论证合理性
项目演示与答辩	5%	演示流畅度、技术问题回答、团队配合

鸿蒙技术栈考核要求

- **必须考核：**所有同学必须参与鸿蒙技术栈实验和考核
- **考核方式：**实验五+期末大作业中均需体现鸿蒙开发成果
- **评分权重：**鸿蒙技术栈成果占技术深度评分的 20%

各等级标准

优秀（90-100 分）：

- 核心功能全部实现且运行稳定
- 各技术栈深度运用平台特性，传感器/物联网集成完善
- **鸿蒙技术栈考核通过**
- AI 新范式四阶段文档完整规范，TRAE 使用记录详实
- Git 协作规范，代码审查记录完整
- 技术选型报告分析深入，论证有力

良好（70-89 分）：

- 核心功能基本实现，运行较稳定
- 各技术栈能运用主要特性，有传感器集成
- 鸿蒙技术栈基本掌握
- AI 新范式流程基本完整，有 TRAE 使用记录

- 能使用 Git 协作，有基本的提交规范
- 技术选型报告内容较完整

合格（60-69 分）：

- 主要功能实现，存在部分缺陷
- 各技术栈基本功能可用
- AI 新范式流程有所体现
- 能进行基本的 Git 操作
- 有简要的技术选型说明

不合格（0-59 分）：

- 核心功能未实现或无法运行
- 技术栈实现不完整
- **未掌握鸿蒙技术栈**
- 未体现 AI 新范式流程
- 无 Git 协作记录
- 缺少技术文档

提交要求清单

必须提交材料

序号	材料	格式要求
1	多端应用源码	Gitee 仓库，含完整提交历史
2	后端 API 文档	含接口定义、数据格式
3	需求分析文档	Markdown，含功能列表、数据模型、API 规范
4	技术选型对比报告	Markdown，各框架优劣分析与选型论证
5	测试报告	测试用例、测试结果、Bug 记录
6	整合测试报告	含跨平台功能测试用例、数据同步验证结果
7	AI 开发过程记录	TRAE 使用截图、辅助编码记录、效率分析
8	项目演示视频	MP4 格式，8-10 分钟，展示各平台功能及跨框架交互效果
9	个人技术总结	每人一份，总结所负责技术栈的学习心得
10	鸿蒙技术栈成果	鸿蒙应用安装包/截图、技术要点总结
11	PlantUML 项目模型图	项目架构图、类图、顺序图等
12	SQLite 数据库设计	数据库表结构、ER 图、数据字典
13	Firebase 云服务配置	用户认证、数据库存储、消息通知配置说明
14	数据同步方案	跨平台数据同步、离线在线数据同步方案说明
15	贡献评估报告	代码质量、功能完善、性能优化等方面的评估

时间节点

阶段	时间	提交内容
需求评审	第 2 天	需求文档 + API 规范 + Gitee 仓库初始化
中期检查	第 7 天	核心功能演示 + 进度报告 + 鸿蒙开发进度
最终提交	第 14 天	全部材料
AI 批阅	第 14 天	项目代码提交后，AI 自动批阅，提供反馈和建议
答辩演示	第 15 天	团队答辩 + 现场演示

 方案实施建议

 成功关键要素

- 1. **团队组建**：技能互补、责任心强、沟通顺畅的团队
- 2. **技术选型**：根据团队技术背景合理分配技术栈，**确保鸿蒙技术栈有人负责**
- 3. **AI 工具善用**：充分利用 TRAE 提升开发效率，但不依赖
- 4. **Gitee 规范**：从第一天起建立规范的分支策略和提交习惯，使用 Gitee 进行版本控制
- 5. **持续沟通**：每日站会同步进度，及时解决问题
- 6. **PlantUML 图表**：使用 PlantUML 绘制项目架构图、类图、顺序图等，确保图表清晰规范
- 7. **SQLite 数据库**：合理设计数据库表结构，确保数据存储和查询效率
- 8. **Firebase 云服务**：正确配置 Firebase 用户认证、数据库存储、消息通知等服务
- 9. **数据同步机制**：实现跨平台数据同步和离线在线数据同步，确保数据一致性

 注意事项

- 避免功能过载：重点关注核心功能的完整实现
- **鸿蒙是必须技术栈**：所有同学必须掌握，建议每组至少 2 人负责鸿蒙开发
- iOS 部分可使用模拟器演示，不强制要求真机运行
- 传感器功能在模拟器上可能受限，可使用模拟数据补充
- 后端 API 可使用教师提供的统一服务或 Mock 数据
- 重视文档质量，AI 新范式四阶段过程记录是重要评分依据
- **鸿蒙技术栈成果作为重要评分项**，包括：ArkUI 页面、传感器调用、多端适配
- **文档简化**：考核文档采用 Markdown 格式为主，PDF 提交，只保留必要内容
- **Gitee 项目仓库**：学生需要在 Gitee 上创建项目仓库，方便查看和管理项目代码
- **贡献评估**：包括代码质量、功能完善、性能优化等方面的评估，减少学生在项目中遇到的问题和挑战
- **AI 批阅环节**：学生提交项目代码后，AI 会自动批阅，提供反馈和建议
- **项目管理环节**：学生需要在项目中进行有效的沟通和合作，确保项目按计划进行
- **项目展示环节**：学生需要在项目完成后，展示项目功能和性能，确保项目符合要求